

Proyecto Aplicación React AEMET:

Iván Escobar Sánchez

PROYECTO

En este proyecto he desarrollado una aplicación meteorológica con arquitectura cliente-servidor. El frontend está hecho con React y la maquetación con Bootstrap, logrando un diseño responsive. El backend está implementado en Node.js + Express y actúa como intermediario entre React y la API oficial de AEMET, de forma que la clave de AEMET nunca se expone en el frontend.

La aplicación permite consultar: predicción por provincia (hoy/mañana), predicción por CCAA (hoy/mañana), observación en tiempo real usando la estación más cercana (por geolocalización o por selección en mapa) y la imagen del radar de lluvias. Además, se muestran rankings tipo top 5 de estaciones con valores máximos/mínimos de diferentes variables.

BACKEND

El backend ha sido desarrollado usando [Node.js](#) y Express. Funciona realizando consultas a la API oficial de AEMET <https://opendata.aemet.es/dist/index.html#/> y devolver datos procesados al frontend en formato JSON.

Para poder realizar estas consultas se necesita una API key que se obtiene solicitándola a AEMET en la siguiente URL <https://opendata.aemet.es/centrodedescargas/altaUsuario>. La API key la he guardado en el archivo .env bajo la variable llamada AEMET_API_KEY, accediendo a ella desde el código mediante process.env.

Las consultas a la api devuelven un JSON con los campos descripción, estado, datos y metadatos. El campo datos contiene una url donde se encuentra el verdadero contenido de la respuesta .

He utilizado router de Express para organizar las consultas. Además, en la carpeta utils, he creado ccaaParser y provinciaParser, para transformar el texto plano que nos devolvía AEMET a JSON, permitiendo así un manejo más manejable en el frontend.

El servidor se configura mediante el archivo [server.js](#) ejecutándose en este caso en el puerto 3000.

Se intentó implementar el funcionamiento de un archivo llamado [alertas.routes.js](#) en el cual se recogían las alertas meteorológicas del país, pero no he podido debido a que me devolvía un archivo comprimido con muchos XML, lo que me imposibilitó su implementación.

Listado de rutas del backend

Ruta main de la API AEMET: <https://opendata.aemet.es/opendata/api/>

GET /observacion/convencional/todas

GET /prediccion/provincia/hoy/:provincia

GET /prediccion/provincia/manana/:provincia

GET /prediccion/ccaa/hoy/:ccaa

GET /prediccion/ccaa/manana/:ccaa

GET /red/radar/nacional

FRONTEND

El frontend ha sido desarrollado con React. Se encarga de la parte visual de la aplicación, realizando llamadas al backend y mostrando los datos obtenidos.

La estructura del proyecto incluye:

- assets: contiene imágenes como el logo de AEMET.
- components: contiene los distintos componentes de la aplicación.
- constants: incluye los diccionarios con nombres y códigos de provincias y CCAA.
- utils: contiene funciones auxiliares como toSlug, que formatea el texto introducido por el usuario para adaptarlo a la URL.

Se han utilizado:

- useState para gestionar estados de datos, carga y error.
- useEffect para ejecutar peticiones al backend cuando el componente se monta o cambian sus dependencias.
- useMemo para optimizar cálculos como la estación más cercana o los rankings top 5.
- react-router-dom para la navegación entre componentes.
- react-leaflet para implementar el mapa interactivo.

- La API de geolocalización del navegador (navigator.geolocation) para obtener la ubicación del usuario.

Dentro de los useEffects, he creado funciones asincronas para envolver el Fetch pudiendo hacer las peticiones al backend de forma async y cargar los datos así. He manejado los errores y los estados de carga creando variables y aplicandoles useState para ir actualizando su estado en cada momento, lanzando alerts de bootstrap en caso de error o spinner en caso de carga, además de bloques try-catch donde lanzan un throw error que llega a nuestro estado de error. He controlado los errores tanto de búsqueda vacía, cómo de provincia/ccaa no encontrada o de fallos de AEMET o del Backend.

MAQUETACIÓN

Para el diseño se ha utilizado Bootstrap, manteniendo una estructura uniforme en todas las páginas con el mismo header y footer.

Se ha aplicado:

- py-4 en la cabecera para añadir padding vertical.
- container my-4 en el contenido principal para mantener márgenes uniformes.
- row g-4 para separar columnas de manera consistente.
- Diseño basado en cards para estructurar la información.
- Spinner para estados de carga y alertas de Bootstrap para mostrar errores.

El diseño es responsive y se adapta correctamente tanto a escritorio como a dispositivos móviles.

INSTALACIONES

Tras implantar la plantilla del Backend con un servidor Express configurado y CORS habilitado, las instalaciones que hay que hacer son:

Backend

- 1- Instalar dependencias
npm install
- 2- Configurar variables de entorno
cp .env.example .env
- 3- Ejecutar en producción
npm start
- 4-Ejecutar en desarrollo
npm run dev

Si no se parte de una plantilla previa:

```
npm install express  
npm install cors  
npm install dotenv  
npm install -D nodemon
```

Frontend

```
1- Para crear un proyecto React  
npm create vite@latest  
    1- Estableces nombre proyecto  
    2- Seleccionas framework: React  
    3- Seleccionas variante: JS + SWC  
Para ejecutar el frontend  
    npm run dev  
Luego como parte del proyecto he descargado  
  
React Router(para las rutas)  
    npm install react-router-dom  
Bootstrap  
    npm install bootstrap  
Leaflet(mapa)  
    npm install leaflet react-leaflet
```

CONCLUSIONES

Como conclusión final, destaco la dificultad de transformar el texto plano devuelto por AEMET en un formato JSON estructurado y fácil de utilizar. No me ha gustado el funcionamiento de la API de AEMET, ya que hay llamadas difíciles de realizar al pedir códigos que no publican y devuelven datos no muy útiles o complejos.

También he tenido problemas al cargar muchas veces una página, me daba un error en la solicitud a AEMET, lo que me hacía tener que recargar la página e incluso a veces, el proyecto.

Por otro lado, me ha resultado con facilidad poder conectar el Backend con el Frontend accediendo a la ruta. De React, me quedo con que es una herramienta sencilla de usar y maquetar, siendo fácil el implementar la lógica y pintarla, sin necesidad de controllers, views o viewsmoel.

Este proyecto cumple con los requisitos mínimos exigidos, incluyendo arquitectura cliente-servidor, gestión de errores, uso de React con hooks y backend en Express, además de mejoras opcionales como mapa interactivo y rankings de estaciones.